

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

**ARYAN MONTI KUMAR JADAWALA
(1BM24CS058)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019**

February to June 2026
CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **ARYAN MONTI KUMAR JADAWALA (1BM24CS058)**, who is Bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	5
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	8
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	11
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	14
6	Implement 0/1 Knapsack problem using dynamic programming.	17
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	19
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	21
9	Implement Fractional Knapsack using Greedy technique.	25
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	27
11	Implement "N-Queens Problem" using Backtracking.	29
12	Leetcode Problems	31

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

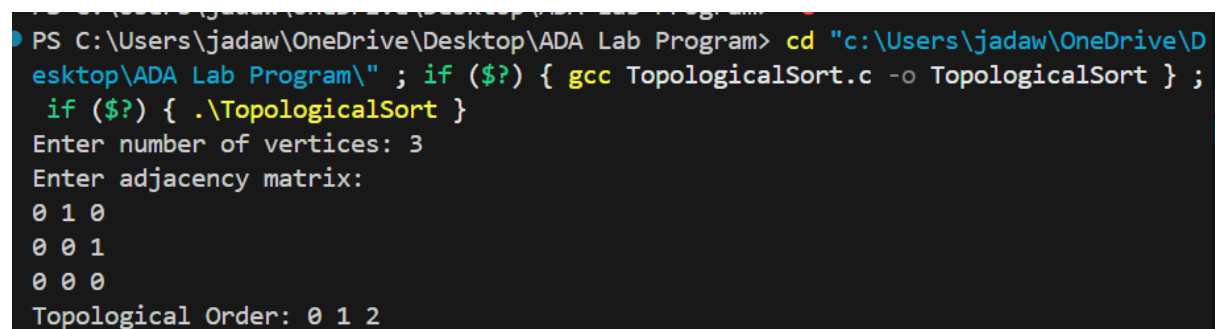
Write program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, i, j, graph[MAX][MAX], indegree[MAX] = {0};
    int queue[MAX], front = 0, rear = -1, topo[MAX], k = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            if (graph[i][j] == 1) indegree[j]++;
        }
    }
    for (i = 0; i < n; i++) if (indegree[i] == 0) queue[++rear] = i;
    while (front <= rear) {
        int v = queue[front++]; topo[k++] = v;
        for (j = 0; j < n; j++) {
            if (graph[v][j] == 1) {
                indegree[j]--;
                if (indegree[j] == 0) queue[++rear] = j;
            }
        }
    }
    printf("Topological Order: ");
    for (i = 0; i < k; i++) printf("%d ", topo[i]);
    return 0;
}
```

OUTPUT:



```
PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab Program\" ; if ($?) { gcc TopologicalSort.c -o TopologicalSort } ;
if ($?) { .\TopologicalSort }
Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
0 0 1
0 0 0
Topological Order: 0 1 2
```

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int val;
    int dir;
    struct Node *next;
} N;
N* create(int n) {
    N *h = NULL, *t = NULL;
    for (int i = 1; i <= n; i++) {
        N *p = (N*)malloc(sizeof(N));
        p->val = i;
        p->dir = 0;
        p->next = NULL;
        if (!h) h = t = p;
        else {
            t->next = p;
            t = p;
        }
    }
    return h;
}
void print(N* h) {
    while (h) {
        printf("%d%s ", h->val, (h->dir == 0 ? "<" : ">"));
        h = h->next;
    }
    printf("\n");
}
N* findMobile(N* h) {
    N *m = NULL;
    N *curr = h;
    N *prev = NULL;
    while (curr) {
        if (curr->dir == 0 && prev != NULL) {
            if (curr->val > prev->val) {
                if (!m || curr->val > m->val) m = curr;
            }
        }
        if (curr->dir == 1 && curr->next != NULL) {
            if (curr->val > curr->next->val) {
                if (!m || curr->val > m->val) m = curr;
            }
        }
        prev = curr;
        curr = curr->next;
    }
    return m;
}
```

```

        curr = curr->next;
    }
    return m;
}

void flipDirections(N* h, int mobileVal) {
    N* curr = h;
    while (curr) {
        if (curr->val > mobileVal) {
            curr->dir = !curr->dir;
        }
        curr = curr->next;
    }
}

void swapNodes(N* a, N* b) {
    int tempVal = a->val;
    int tempDir = a->dir;
    a->val = b->val;
    a->dir = b->dir;
    b->val = tempVal;
    b->dir = tempDir;
}

int main() {
    int n;
    printf("Enter the number of elements (n): ");
    scanf("%d", &n);
    N* h = create(n);
    printf("\nStarting Permutation:\n");
    print(h);
    N* m;
    while ((m = findMobile(h)) != NULL) {
        int mobileVal = m->val;
        if (m->dir == 0) {
            N* prev = h;
            while (prev->next != m) prev = prev->next;
            swapNodes(m, prev);
        } else {
            swapNodes(m, m->next);
        }
        flipDirections(h, mobileVal);
        print(h);
    }

    return 0;
}

```

OUTPUT:

```
PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab Program\" ; if ($?) { gcc JohnsonTrotter.c -o JohnsonTrotter } ; if ($?) { .\JohnsonTrotter }
Enter the number of elements (n): 3

Starting Permutation:
1<- 2<- 3<-
1<- 3<- 2<-
3<- 1<- 2<-
3-> 2<- 1<-
2<- 3-> 1<-
2<- 1<- 3->
```

Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void merge(int arr[], int left, int mid, int right)
{
    int temp[50];
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right)
    {
        if (arr[i] < arr[j])
        {
            temp[k++] = arr[i++];
        }
        else
        {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid)
    {
        temp[k++] = arr[i++];
    }

    while (j <= right)
    {
        temp[k++] = arr[j++];
    }

    for (i = left; i <= right; i++)
    {
        arr[i] = temp[i];
    }
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;
```



```

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

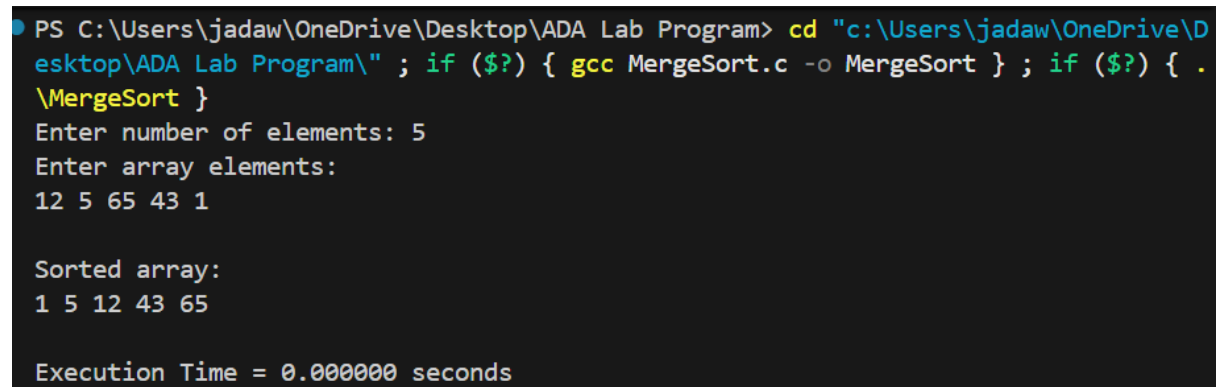
    int arr[n];
    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    clock_t start, end;
    double cpu_time_used;

    start = clock();
    mergeSort(arr, 0, n - 1);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n\nExecution Time = %f seconds\n", cpu_time_used);
    return 0;
}

```

OUTPUT:



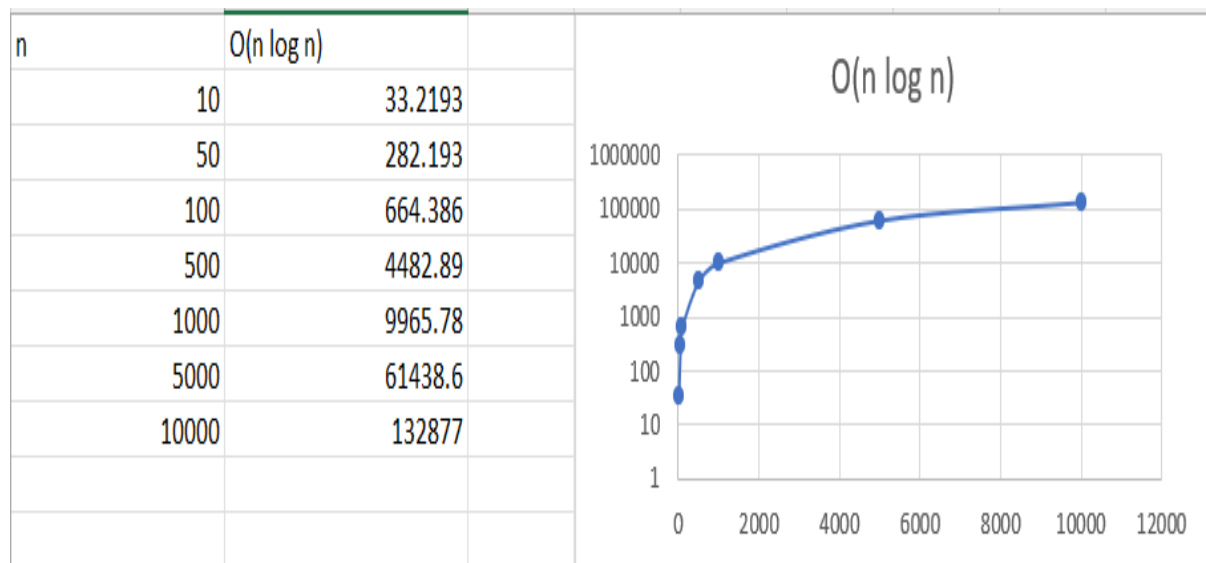
```

PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab Program" ; if ($?) { gcc MergeSort.c -o MergeSort } ; if ($?) { . \MergeSort }
Enter number of elements: 5
Enter array elements:
12 5 65 43 1

Sorted array:
1 5 12 43 65

Execution Time = 0.000000 seconds

```



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <time.h>

void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quick_sort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);

        quick_sort(arr, low, p - 1);
        quick_sort(arr, p + 1, high);
    }
}

int main()
{
    int n;
```

```

printf("Enter size of array: ");
scanf("%d", &n);

int arr[n];

printf("Enter array elements:\n");

for (int i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
}

clock_t start, end;
start = clock();
quick_sort(arr, 0, n - 1);
end = clock();
double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;
printf("\nSorted array:\n");
for (int i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

printf("\n\nTime Taken = %.6f ms\n", time_taken);

return 0;
}

```

Output:

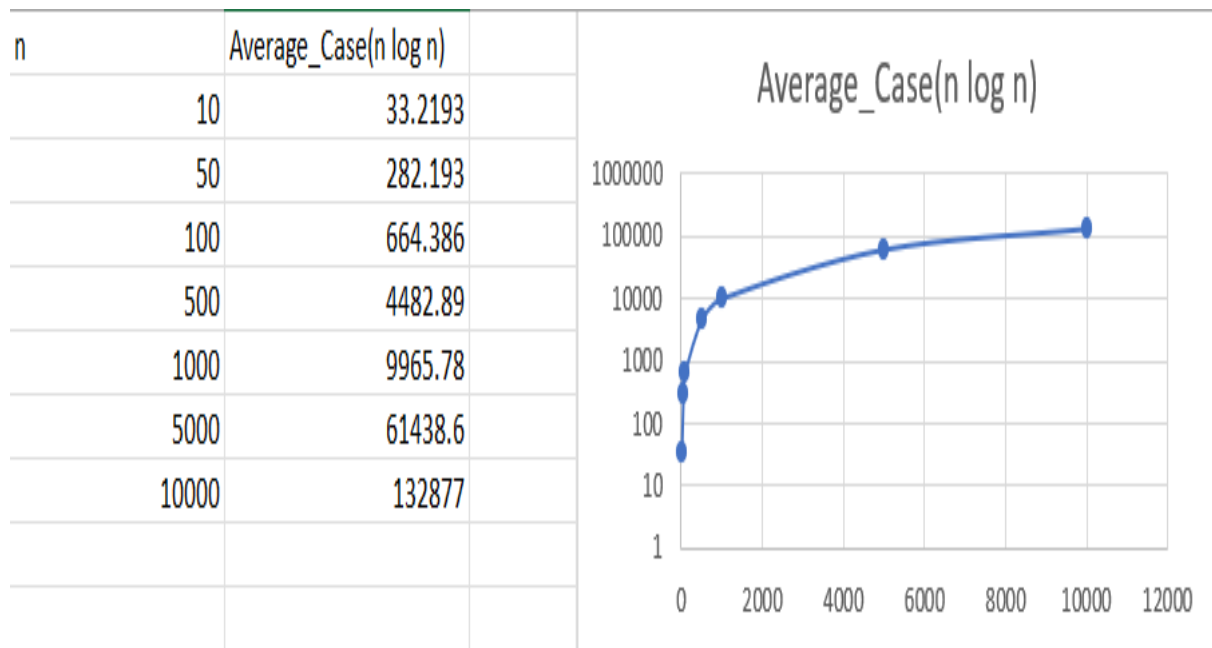
```

PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab
Program\" ; if ($?) { gcc QuickSort.c -o QuickSort } ; if ($?) { .\QuickSort }
Enter size of array: 6
Enter array elements:
1 19 15 97 0 3

Sorted array:
0 1 3 15 19 97

Time Taken = 0.000000 ms

```



Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <time.h>

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    int i, temp;

    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n - 1; i > 0; i--)
    {
        temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}
```

```

void main()
{
    int n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

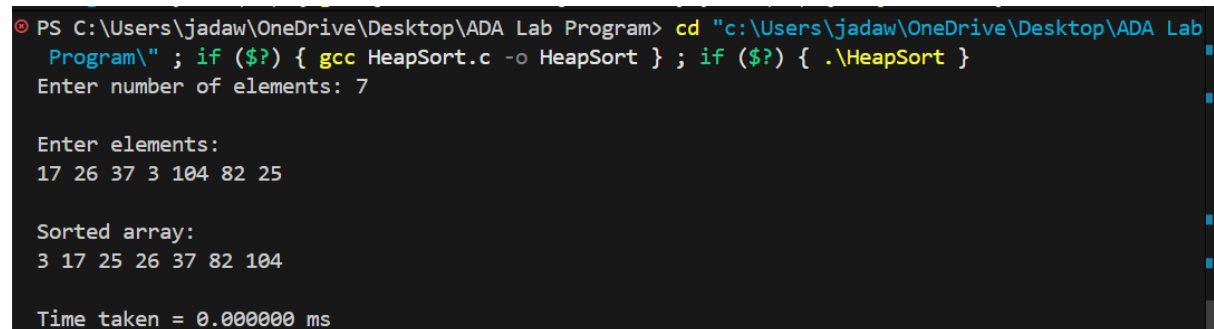
    int arr[n];
    printf("\nEnter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    start = clock();
    heapSort(arr, n);
    end = clock();
    printf("\nSorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    time_taken = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;

    printf("\n\nTime taken = %f ms\n", time_taken);
}

```

Output:



```

PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab
Program\" ; if ($?) { gcc HeapSort.c -o HeapSort } ; if ($?) { .\HeapSort }
Enter number of elements: 7

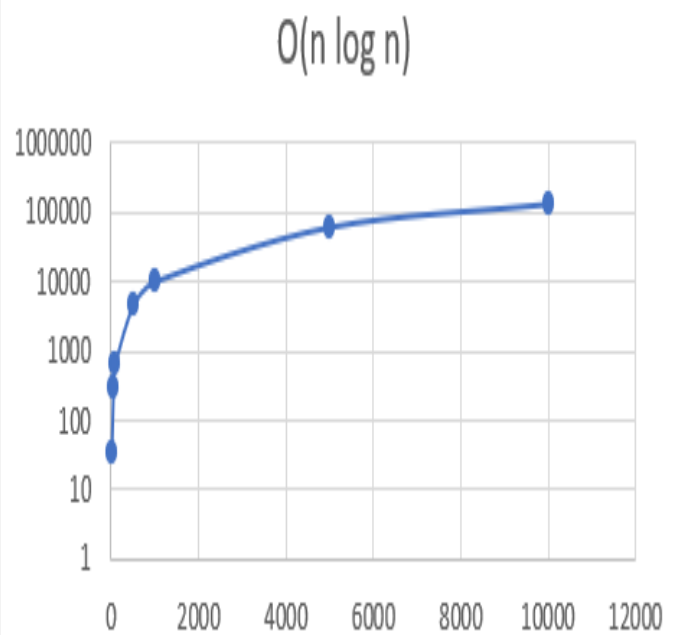
Enter elements:
17 26 37 3 104 82 25

Sorted array:
3 17 25 26 37 82 104

Time taken = 0.000000 ms

```

n	$O(n \log n)$	
10	33.2193	
50	282.193	
100	664.386	
500	4482.89	
1000	9965.78	
5000	61438.6	
10000	132877	



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>

int main()
{
    int n, w[50], p[50], m, a[50][50];
    int i, j;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the maximum capacity: ");
    scanf("%d", &m);
    printf("Enter the weights: ");
    for (i = 1; i <= n; i++)
    {
        scanf("%d", &w[i]);
    }
    printf("Enter the profits: ");
    for (i = 1; i <= n; i++)
    {
        scanf("%d", &p[i]);
    }
    for (i = 0; i <= n; i++)
        a[i][0] = 0;
    for (j = 0; j <= m; j++)
        a[0][j] = 0;
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= m; j++)
        {
            if (j < w[i])
            {
                a[i][j] = a[i - 1][j];
            }
            else
            {
                if (a[i - 1][j] > (p[i] + a[i - 1][j - w[i]]))
                {
                    a[i][j] = a[i - 1][j];
                }
                else
                {
                    a[i][j] = p[i] + a[i - 1][j - w[i]];
                }
            }
        }
    }
}
```

```
}  
printf("Total profit: %d\n", a[n][m]);  
return 0;  
}
```

Output:

```
PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab  
Program\" ; if ($?) { gcc FractionalKnapsack.c -o FractionalKnapsack } ; if ($?) { .\Fractiona  
lKnapsack }  
Enter the number of elements: 4  
Enter the maximum capacity: 5  
Enter the weights: 2 1 3 2  
Enter the profits: 12 10 20 15  
Total profit: 37
```

Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

Code:

```
#include <stdio.h>

void Floyd's(int n, int cost[50][50], int A[50][50])
{
    int i, j, k;

    for(i = 1; i <= n; i++)
        for(j = 1; j <= n; j++)
            A[i][j] = cost[i][j];

    for(k = 1; k <= n; k++)
        for(i = 1; i <= n; i++)
            for(j = 1; j <= n; j++)
                if(A[i][k] + A[k][j] < A[i][j])
                    A[i][j] = A[i][k] + A[k][j];
}

int main() {
    int n, i, j;
    int cost[50][50], A[50][50];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix (use 999 for infinity):\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j) {
                cost[i][j] = 999;
            }
        }
    }
    Floyd's(n, cost, A);
    printf("\nThe all-pairs shortest path matrix is:\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            if (A[i][j] == 999) printf("INF ");
            else printf("%d ", A[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA
Program\" ; if ($?) { gcc Floyd.c -o Floyd } ; if ($?) { .\Floyd }
Enter number of vertices: 4
Enter the cost matrix (use 999 for infinity):
0 999 6 1
4 0 20 10
999 3 0 12
6 999 999 0

The all-pairs shortest path matrix is:
0 9 6 1
4 0 10 5
7 3 0 8
6 15 12 0
```

Lab Program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Code:

```
#include <stdio.h>

#define INF 999
#define MAX 50

void prims(int n, int cost[MAX][MAX]) {
    int parent[MAX], key[MAX], mstSet[MAX];
    int i, v, count, u, min, totalcost = 0;

    for (i = 0; i < n; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (count = 0; count < n - 1; count++) {
        min = INF;
        for (v = 0; v < n; v++) {
            if (mstSet[v] == 0 && key[v] < min) {
                min = key[v];
                u = v;
            }
        }

        mstSet[u] = 1;

        for (v = 0; v < n; v++) {
            if (cost[u][v] != 0 && mstSet[v] == 0 && cost[u][v] < key[v]) {
                parent[v] = u;
                key[v] = cost[u][v];
            }
        }
    }

    printf("\nEdge \tWeight\n");
    for (i = 1; i < n; i++) {
        printf("%d - %d \t %d\n", parent[i], i, cost[i][parent[i]]);
        totalcost += cost[i][parent[i]];
    }
    printf("\nTotal minimum Cost: %d\n", totalcost);
}
```

```

int main() {
    int n, cost[MAX][MAX];
    printf("Enter no of vertices: ");
    scanf("%d", &n);

    printf("Enter Adjacency matrix (use 999 for no edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    prims(n, cost);

    return 0;
}

```

Output:

```

PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab Program\" ; if ($?) { gcc Prisms.c -o Prisms } ; if ($?) { .\Prisms }
Enter no of vertices: 4
Enter Adjacency matrix (use 999 for no edge):
0 2 999 6
2 0 3 8
999 3 0 0
6 8 0 0

Edge      Weight
0 - 1      2
1 - 2      3
0 - 3      6

Total minimum Cost: 11

```

Lab Program 8 b:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code:

```
#include <stdio.h>

struct edge {
    int s, e, cost;
};

int cost = 0;
int grp[100];

int find(int x) {
    if (x == grp[x])
        return x;
    return grp[x] = find(grp[x]);
}

void merge(int x, int y, int c) {
    int gx = find(x);
    int gy = find(y);
    if (gx != gy) {
        grp[gy] = gx;
        cost += c;
    }
}

int main() {
    struct edge a[100];
    int n;

    printf("Enter the number of edges: ");
    scanf("%d", &n);

    printf("Enter src, dest and cost for following\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i].s);
        scanf("%d", &a[i].e);
        scanf("%d", &a[i].cost);
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i].cost > a[j].cost) {
                struct edge t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }

    for (int i = 0; i < n; i++) {
        int x = find(a[i].s);
        int y = find(a[i].e);
        if (x != y) {
            merge(a[i].s, a[i].e, a[i].cost);
        }
    }

    printf("Minimum Cost Spanning Tree Cost: %d", cost);
}
```

```

        a[j] = t;
    }
}
}

for (int i = 0; i < 100; i++) {
    grp[i] = i;
}
for (int i = 0; i < n; i++) {
    merge(a[i].s, a[i].e, a[i].cost);
}
printf("\nFor the total cost is: %d\n", cost);
return 0;
}

```

Output:

```

PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA
Program\" ; if ($?) { gcc Krushkals.c -o Krushkals } ; if ($?) { .\Krushkals }
Enter the number of edges: 5
Enter src, dest and cost for following
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

For the total cost is: 19

```


Lab Program 9:

Implement fractional knapsack problem using Greedy technique.

Code:

```
#include<stdio.h>
struct item{
    int p;
    int w;
    float r;
    float x;
};
int main(){
    int n, m;
    printf("Enter the number of items: ");
    scanf("%d", &n);
    printf("Enter the maximum capacity: ");
    scanf("%d", &m);
    struct item items[20], temp;
    for(int i=0; i<n; i++){
        printf("Enter weight of item %d: ", i+1);
        scanf("%d", &items[i].w);
        printf("Enter profit of item %d: ", i+1);
        scanf("%d", &items[i].p);
        items[i].x = 0;
        items[i].r = (float)items[i].p / items[i].w;
    }
    for(int i=0; i<n; i++){
        for(int j=i+1; j<n; j++){
            if(items[j].r > items[i].r){
                temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
    float remain = m, maxprofit = 0;
    for(int i=0; i<n; i++){
        if(items[i].w <= remain){
            items[i].x = 1;
            remain -= items[i].w;
            maxprofit += items[i].p;
        } else {
            items[i].x = remain / items[i].w;
            maxprofit += items[i].p * items[i].x;
            break;
        }
    }
}
```

```
printf("\nMaximum profit = %.2f\n", maxprofit);  
}
```

Output:

```
PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA  
Program\" ; if ($?) { gcc FractionalKnapsack.c -o FractionalKnapsack } ; if ($?) { .\Fract  
lKnapsack }  
Enter the number of items: 3  
Enter the maximum capacity: 50  
Enter weight of item 1: 13  
Enter profit of item 1: 34  
Enter weight of item 2: 20  
Enter profit of item 2: 54  
Enter weight of item 3: 40  
Enter profit of item 3: 90  
  
Maximum profit = 126.25
```

Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code

```
#include <stdio.h>

int main() {
    int n, cost[20][20], dist[20];
    int visited[20]; // Changed 's' to 'visited' to match your logic
    int i, j, source, u, min;

    printf("Enter number of vertices\n");
    scanf("%d", &n);

    printf("Enter the cost matrix\n");
    for(i=1; i<=n; i++) {
        for(j=1; j<=n; j++) {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0 && i != j)
                cost[i][j] = 999;
        }
    }

    printf("Enter source vertex\n");
    scanf("%d", &source);

    for(i=1; i<=n; i++) {
        dist[i] = cost[source][i];
        visited[i] = 0;
    }

    visited[source] = 1;
    dist[source] = 0;

    for(i=1; i<=n; i++) {
        min = 999;
        for(j=1; j<=n; j++) {
            if(!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }
        visited[u] = 1;
        for(j=1; j<=n; j++) {
            if(!visited[j] && (min + cost[u][j] < dist[j])) {
                dist[j] = min + cost[u][j];
            }
        }
    }
}
```

```

    }
}

printf("Shortest distances from vertex %d:\n", source);
for(i=1; i<=n; i++) {
    printf("to vertex %d = %d\n", i, dist[i]);
}

return 0;
}

```

Output:

```

PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab Program\" ; if ($?) { gcc Djikstras.c -o Djikstras } ; if ($?) { .\Djikstras }
Enter number of vertices
4
Enter the cost matrix
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Enter source vertex
0
Shortest distances from vertex 0:
to vertex 1 = 0
to vertex 2 = 0
to vertex 3 = 0
to vertex 4 = 0

```

Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int x[20], n;
int solutionCount = 0;

int placeQueen(int k, int i){
    for (int j = 1; j <= k - 1; j++)
    {
        if ((x[j] == i) || (abs(j - k) == abs(x[j] - i)))
            return 0;
    }
    return 1;
}

void printSolution(){
    solutionCount++;
    printf("\nSolution %d:\n", solutionCount);
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (x[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
}

void nQueens(int k){
    for (int i = 1; i <= n; i++)
    {
        if (placeQueen(k, i))
        {
            x[k] = i;
```

```

        if (k == n)
            printSolution();
        else
            nQueens(k + 1);
    }
}

void main(){
    clock_t start, end;

    printf("Enter number of queens: ");
    scanf("%d", &n);

    start = clock();
    nQueens(1);
    end = clock();
    if (solutionCount == 0)
        printf("\nNo Solution Exists\n");
    else
        printf("\nTotal Solutions = %d\n", solutionCount);

    double time_taken =
        (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;

    printf("Time Taken: %f ms\n", time_taken);
}

```

Output:

```

PS C:\Users\jadaw\OneDrive\Desktop\ADA Lab Program> cd "c:\Users\jadaw\OneDrive\Desktop\ADA Lab
Program\" ; if ($?) { gcc NQueens.c -o NQueens } ; if ($?) { .\NQueens }
Enter number of queens: 4

Solution 1:
.  Q  .  .
.  .  .  Q
Q  .  .  .
.  .  Q  .

Solution 2:
.  .  Q  .
Q  .  .  .
.  .  .  Q
.  Q  .  .

Total Solutions = 2
Time Taken: 5.000000 ms

```

LEETCODE 34. Find First and Last Position of Element in Sorted Array

Program:

```
#include <stdlib.h>
```

```
int* searchRange(int* nums, int numsSize, int target, int* returnSize) {  
    *returnSize = 2;
```

```
  
    int* result = (int*)malloc(2 * sizeof(int));  
    result[0] = -1;  
    result[1] = -1;
```

```
  
    for (int i = 0; i < numsSize; i++) {  
        if (nums[i] == target) {  
            result[0] = i;  
            break;  
        }  
    }
```

```
    for (int i = numsSize - 1; i >= 0; i--) {  
        if (nums[i] == target) {  
            result[1] = i;  
            break;  
        }  
    }
```

```
    return result;  
}
```

Output:

The screenshot displays the LeetCode submission interface for the problem "Find First and Last Position of Element in Sorted Array". The interface is divided into several sections:

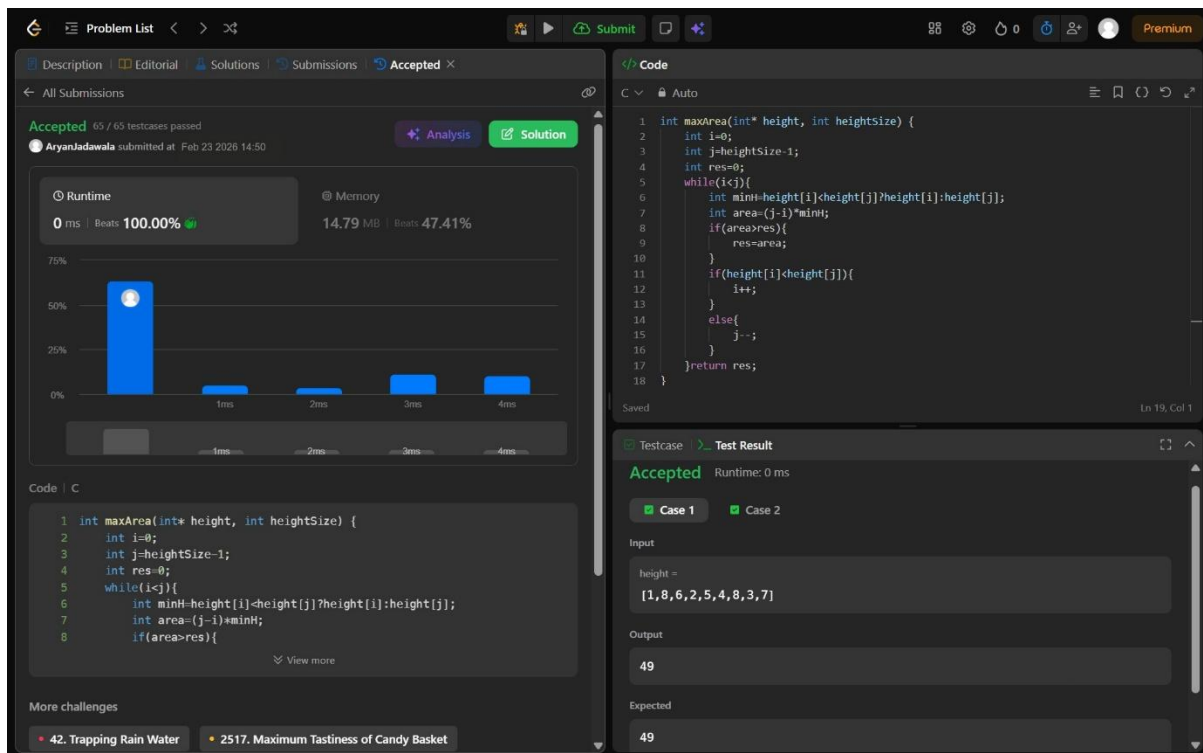
- Problem List:** Shows the problem name and a "Premium" badge.
- Accepted:** Indicates that the solution is accepted, with 88/88 test cases passed. The submission was made by "AryanJadav" on Feb 23, 2026, at 15:00.
- Runtime and Memory:** The runtime is 0 ms, which is 100.00% faster than the average. The memory usage is 10.58 MB, which is 34.97% less than the average.
- Code:** The C code is displayed, showing the implementation of the searchRange function. It uses a linear search approach, iterating from the start and end of the array to find the first and last positions of the target.
- Testcase:** The test results are shown, indicating that all three cases (Case 1, Case 2, and Case 3) are accepted. The input for Case 1 is "nums = [5,7,7,8,8,10]" and "target = 8". The output is "0 ms".

LEETCODE 11. Container With Most Water

Program:

```
int maxArea(int* height, int heightSize) {
    int i=0;
    int j=heightSize-1;
    int res=0;
    while(i<j){
        int minH=height[i]<height[j]?height[i]:height[j];
        int area=(j-i)*minH;
        if(area>res){
            res=area;
        }
        if(height[i]<height[j]){
            i++;
        }
        else{
            j--;
        }
    }return res;
}
```

Output:



LEETCODE 268. Missing Number

Program:

```
int missingNumber(int* nums, int numsSize) {
    int i, j;

    for(i = 0; i < numsSize - 1; i++) {
        for(j = 0; j < numsSize - i - 1; j++) {
            if(nums[j] > nums[j + 1]) {
                int temp = nums[j];
                nums[j] = nums[j + 1];
                nums[j + 1] = temp;
            }
        }
    }

    for(i = 0; i < numsSize; i++) {
        if(nums[i] != i) {
            return i;
        }
    }

    return numsSize;
}
```

Output:

The screenshot displays the LeetCode problem page for 'Missing Number' (Problem 268). The interface is divided into several sections:

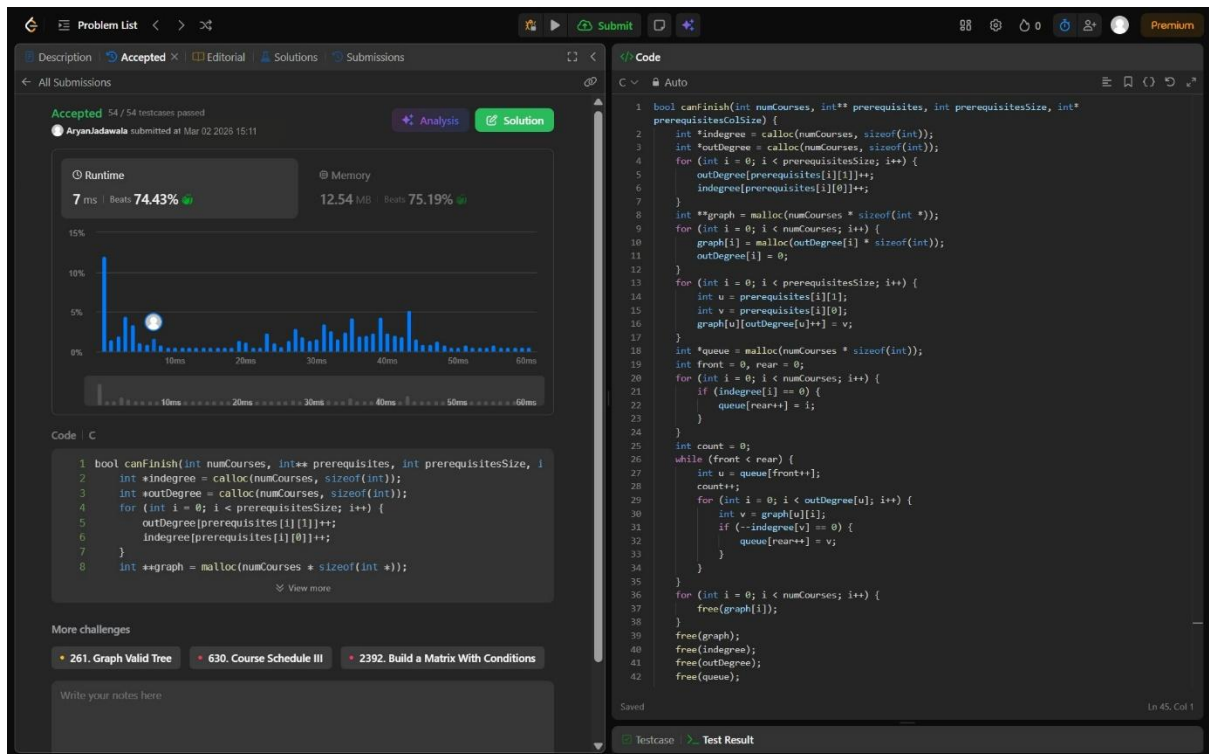
- Problem List:** Shows the problem name and a 'Premium' badge.
- Description:** Contains the problem statement and a 'Solution' button.
- Accepted:** Indicates that 122 out of 122 testcases passed. A submission by 'AryanJadawala' is shown, submitted at Mar 02 2026 14:12.
- Runtime:** Shows a runtime of 0 ms, which beats 100.00% of submissions. A graph shows the runtime performance relative to other submissions.
- Memory:** Shows a memory usage of 9.74 MB, which beats 10.78% of submissions.
- Code:** Displays the C code solution for the problem, which calculates the expected sum of numbers from 0 to n and subtracts the actual sum of the array elements.
- Testcase:** Shows the test result for 'Case 1', 'Case 2', and 'Case 3', all of which are 'Accepted'. The input for Case 1 is 'nums = [3, 0, 1]' and the output is '2'.
- More challenges:** Lists other problems to solve, including '41. First Missing Positive' and '287. Find the Duplicate Number'.

LEETCODE 207. Course Schedule

Program:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int *indegree = calloc(numCourses, sizeof(int));
    int *outDegree = calloc(numCourses, sizeof(int));
    for (int i = 0; i < prerequisitesSize; i++) {
        outDegree[prerequisites[i][1]]++;
        indegree[prerequisites[i][0]]++;
    }
    int **graph = malloc(numCourses * sizeof(int *));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = malloc(outDegree[i] * sizeof(int));
        outDegree[i] = 0;
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int u = prerequisites[i][1];
        int v = prerequisites[i][0];
        graph[u][outDegree[u]++] = v;
    }
    int *queue = malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    int count = 0;
    while (front < rear) {
        int u = queue[front++];
        count++;
        for (int i = 0; i < outDegree[u]; i++) {
            int v = graph[u][i];
            if (--indegree[v] == 0) {
                queue[rear++] = v;
            }
        }
    }
    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }
    free(graph);
    free(indegree);
    free(outDegree);
    free(queue);
    return count == numCourses;
}
```

Output:



LEETCODE 801. Minimum Swaps To Make Sequences Increasing

Program:

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swap = 1;
    for (int i = 1; i < nums1Size; i++) {
        int nextKeep = nums1Size + 1;
        int nextSwap = nums1Size + 1;
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
            nextKeep = keep;
            nextSwap = swap + 1;
        }
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
            if (swap < nextKeep)
                nextKeep = swap;
            if (keep + 1 < nextSwap)
                nextSwap = keep + 1;
        }
        keep = nextKeep;
        swap = nextSwap;
    }
    return keep < swap ? keep : swap;
}
```

Output:

The screenshot shows the LeetCode submission interface for problem 801. The left panel displays the submission status as 'Accepted' with a runtime of 4ms and memory usage of 16.40 MB. The right panel shows the C code and the test result for Case 1, which is 'Accepted' with a runtime of 0ms. The input arrays are [1, 3, 5, 4] and [1, 2, 3, 7], and the output is 1.

Submission Details:

- Status: Accepted (117 / 117 testcases passed)
- Runtime: 4 ms (Beats 30.86%)
- Memory: 16.40 MB (Beats 46.91%)

Code:

```
1 int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
2     int keep = 0;
3     int swap = 1;
4     for (int i = 1; i < nums1Size; i++) {
5         int nextKeep = nums1Size + 1;
6         int nextSwap = nums1Size + 1;
7         if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
8             nextKeep = keep;
9             nextSwap = swap + 1;
10        }
11        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
12            if (swap < nextKeep)
13                nextKeep = swap;
14            if (keep + 1 < nextSwap)
15                nextSwap = keep + 1;
16        }
17        keep = nextKeep;
18        swap = nextSwap;
19    }
20    return keep < swap ? keep : swap;
21 }
22
```

Test Result:

- Accepted (Runtime: 0 ms)
- Case 1: Accepted
- Case 2: Accepted

Input:

```
nums1 = [1, 3, 5, 4]
nums2 = [1, 2, 3, 7]
```

Output:

```
1
```

LEETCODE 287. Find the Duplicate Number

Program:

```
int findDuplicate(int* nums, int numsSize) {

    int slow = nums[0];
    int fast = nums[0];

    do{
        slow = nums[slow];
        fast = nums[nums[fast]];
    }while (slow !=fast);

    slow = nums[0];
    while (slow !=fast){
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}
```

Output:

The screenshot displays the LeetCode interface for problem 287. The top section shows the problem title and a submission status of 'Accepted' with 59/59 test cases passed. The submission was made by 'AryanJadawala' on May 30, 2026, at 14:51. The runtime is 4 ms, beating 62.74% of solutions, and the memory usage is 14.88 MB, beating 81.37%. A bar chart shows the distribution of runtime times, with most solutions falling between 10ms and 30ms. The code editor on the right shows the C code for the 'findDuplicate' function. The test result panel at the bottom right shows the input array [1, 3, 4, 2, 2] and the output 2, which matches the expected result.

```
1 int findDuplicate(int* nums, int numsSize) {
2     int slow = nums[0];
3     int fast = nums[0];
4
5     do{
6         slow = nums[slow];
7         fast = nums[nums[fast]];
8     }while (slow !=fast);
9
10    slow = nums[0];
11    while (slow !=fast){
12        slow = nums[slow];
13        fast = nums[fast];
14    }
15
16    return slow;
17 }
18
19
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =

[1,3,4,2,2]

Output

2

Expected